

Manual Técnico — Pipeline de Autorizaciones Intrahospitalarias (ERP-84)

Guía paso a paso de **cómo funciona** el pipeline de autorizaciones y **qué configuración** hay que dejar lista para ponerlo funcional en un ambiente (QA / nuevo tenant). Incluye el nuevo **Agente Autorizador** (distribución automática de usuarios autorizadores a ingresos).

Regla de oro que atraviesa todo el documento: **crear una tabla, un topic o habilitar Change Tracking NO otorga permisos**. Cada Managed Identity necesita sus GRANTS / roles RBAC explícitos **por ambiente y por tenant**. El 90% de las fallas de este pipeline fueron permisos faltantes.

Índice

1. Qué es

2. Arquitectura — los 4 flujos

3. Recursos del ambiente (para DevOps)

4. Configuración para ponerlo funcional (paso a paso)

4.0 Prerequisito: esquema y tablas desplegadas (DB project / scripts)

4.1 Service Bus

4.2 Change Tracking (base del tenant)

4.3 Grants SQL por Managed Identity (EL PUNTO CRÍTICO)

4.4 Control Plane (registro del tenant)

4.5 App settings (Function Apps)

4.6 Datos semilla del Agente Autorizador (sin esto NO asigna)

5. Verificación end-to-end

6. Troubleshooting (síntoma → causa → fix)

7. Referencias

1. Qué es

Un pipeline event-driven que, ante hechos clínicos (órdenes médicas, atención inicial de urgencias, estancias, ingresos hospitalarios), materializa registros de control en `Authorization.AuthorizationControl` para el Dashboard de Autorizaciones, y **asigna automáticamente un usuario autorizador** al ingreso.

Está compuesto por 3 Azure Functions (.NET 8 isolated) desacopladas por Service Bus:

Function App (ERP-84 dev/QA)	Rol
<code>ehrc-relay-dev-erp84d</code>	Relay — drena las tablas Outbox del tenant (Change Tracking) y publica a <code>ehr-source-events</code> .
<code>ehrc-normalizer-dev-erp84d</code>	Normalizer — lee la HC/contexto una sola vez, enriquece (Cosmos claim-check para órdenes) y publica los <i>facts</i> a los topics de salida.
<code>ehrc-authz-dev-erp84d</code>	AuthorizationControl — consume los facts y escribe el control + (agente) la asignación de autorizador.

2. Arquitectura — los 4 flujos

Todos comparten Relay → `ehr-source-events` → Normalizer, y se ramifican por `eventType`:

Tabla fuente (tenant)	Outbox Consumer (Authz)	eventType Resultado	Topic de salida
HCORD*/HCPRESCRA/... facts	Clinical.OutboxEvent AuthorizationOrderConsumer	clinical.record-committed.v1 AuthorizationControl (SubjectType 1/2/3)	clinical-order-facts
ADATEINIUI facts	Clinical.OutboxEvent AuthorizationEmergencyInitialCare	clinical.record-committed.v1 AuthorizationControl (SubjectType 4 = URGENCIA)	clinical-emergency-facts
CHREGESTA / ACS facts	Hospitalization/ AuthorizationStayConsumer	hospitalization.stay-changed.v1 AuthorizationControl (SubjectType 5 = ESTANCIA)	clinical-stay-facts
	Billing.OutboxEvent	billing.stay-committed.v1	
HCHISPACA.INDICAPAC facts	Clinical.OutboxEvent AuthorizationStayConsumer	clinical.record-committed.v1 AuthorizationControl (SubjectType 5, orden de traslado)	clinical-stay-facts
ADINGRESO facts	Admissions.OutboxEvent AuthorizationAdmissionDistribution	clinical.admission-*.v1 AdmissionAuthorizerAssignmentLog → (trigger)	clinical-admission-facts
	AuthorizationControl.AssignedAuthorizer		

Flujo de órdenes: el Normalizer lee la HC, hace *upsert* del documento desnormalizado en **Cosmos** (claim-check) y publica `clinical.order-normalized.v1`. El consumer resuelve susceptibilidad y crea un control por orden.

Flujo de urgencias (AIU): control `SubjectType=4`, `SourceTable='ADATEINIUI'`, `SubjectRecordId=CODCONCEC`, **RequestedQuantity=1 por defecto** (no se deriva de órdenes).

Flujo de estancias (Stella): billing extraído a su propia función/suscripción (`BillingStayNormalizer` / `billing-stay`); publica a `clinical-stay-facts`. Stella es la única que **acumula** la cantidad de días (`RequestedQuantity` = días reconocidos) y actualiza cama/tipo/fecha sobre la estancia vigente del ingreso.

Flujo de estancia desde HC (orden de traslado, 2026-08): cuando el médico indica un traslado (`HCHISPACA.INDICAPAC` $\in$ {3,4,5,6,9,21}), el mismo `clinical.record-committed.v1` publica —además de las órdenes— un fact de estancia "solo-tipificación" que crea/actualiza la estancia (SubjectType 5) con la orden de traslado. Búsqueda por ingreso (una estancia abierta por ingreso): mismo tipo → actualiza; tipo distinto → borra la anterior y crea la nueva. Los días de estancia NO se calculan aquí (los pone Stella). **Sin objetos de BD nuevos.** Alerta de "estancia no liquidada" (evento futuro de facturación) → `Authorization.AuthorizationControlAlert`.

Flujo del Agente Autorizador (nuevo):

```
dbo.ADINGRESO —(trigger TR_ADINGRESO_AdmissionStatusChanged / emit C#)→ Admissions.OutboxEvent
  ▶ Relay ▶ ehr-source-events ▶ Normalizer (rama admisión 1a2) ▶ clinical-admission-facts
  ▶ AuthorizationAdmissionDistributionConsumer ▶ Authorization.AdmissionAuthorizerAssignmentLog
  ▶ (trigger de backfill) ▶ AuthorizationControl.AssignedAuthorizer
```

Lógica de asignación: **menor carga** — del pool de `UsersAssignment` (activos, hospitalarios) excluyendo los que tengan novedad vigente (`UserNovelties`), se asigna al de menos ingresos vigentes. Interruptor global: `AuthorizationManagementParametersAutomaticAssignment`.

3. Recursos del ambiente (para DevOps)

Cuidado: **los recursos están repartidos en DOS suscripciones distintas** — las Function Apps y Cosmos en *VIE Development*, y el Service Bus en *Indira Development*. El RBAC cruzado (Function App de una suscripción → Service Bus de otra) es el punto que más se olvida.

3.1 Function Apps (las 3 del pipeline)

Function App	Rol	Managed Identity
<code>ehrc-relay-dev-erp84d</code>	Relay	System-Assigned (nombre = el de la app)
<code>ehrc-normalizer-dev-erp84d</code>	Normalizer	System-Assigned
<code>ehrc-authz-dev-erp84d</code>	AuthorizationControl	System-Assigned

- **Suscripción:** VIE Development · **Resource Group:** `rg-erp84-clinical-dev`
- **Runtime:** .NET 8 isolated (Linux). **Identidad:** System-Assigned Managed Identity en cada una (es el "usuario" que recibe todos los grants SQL y roles RBAC de este manual).
- **Key Vault:** `ehrcokvdeverp84d` (mismo RG) — guarda las cadenas SQL; cada MI necesita rol *Key Vault Secrets User* sobre él.

3.2 Service Bus (COMPARTIDO — otra suscripción)

Dato	Valor
Namespace	<code>global-multi-fhir-dev</code>
Suscripción	Indira Development
Resource Group	<code>global-coral-interoperability-service</code>

- Es **infraestructura compartida** — no se re-crea el namespace, solo se agregan topics/subs (§4.1).
- **RBAC cross-subscription:** las MI de las 3 Function Apps (que viven en VIE Development) necesitan roles *Data Sender/Receiver* sobre topics/colas de este

namespace (que vive en Indira Development). Quien asigne ese RBAC necesita permisos en **ambas** suscripciones (o coordinar con el dueño del namespace).

3.3 Cosmos DB

Dato	Valor
Cuenta	<code>global-multi-fhir-dev-cosmos</code> (serverless)
Container	<code>NormalizedMedicalOrders</code> (pk <code>/partitionKey</code>)
Suscripción	VIE Development (confirmar RG exacto con DevOps)

- Rol **data-plane** de Cosmos para Normalizer (Data Contributor) y Authz (Data Reader) — ver §4.1.

3.4 Bases de datos SQL

Base	Qué es
Control Plane (schema <code>Platform</code>)	Catálogo de tenants + cursores. Ej. <code>INDIGOSEC</code> / <code>INDIGOSECV2</code> (server <code>ssindigodev</code>).
Base del tenant	La BD clínica del cliente. Ej. <code>INDIGO636</code> . Es donde viven las tablas <code>outbox</code> , <code>Authorization.*</code> y los triggers.

3.5 Checklist para DevOps (qué hay que provisionar/configurar)

1. Las 3 Function Apps con **System-Assigned Managed Identity** activada.
2. Rol *Key Vault Secrets User* de cada MI sobre `ehrcokvdeverp84d`, y cargar los secretos (cadenas SQL — ver §4.5).
3. Topics + suscripciones + filtros en el Service Bus (§4.1).
4. **RBAC Service Bus cross-subscription** de cada MI (§4.1) — el paso que cruza las dos suscripciones.
5. **RBAC data-plane de Cosmos** para Normalizer/Authz (§4.1).
6. Grants SQL de cada MI en la base del tenant y en el Control Plane (§4.3, §4.4) — los ejecuta el DBA con un admin AAD del servidor SQL.
7. App settings de cada Function App (§4.5).

4. Configuración para ponerlo funcional (paso a paso)

Ejecutar en este orden. Reemplazar `<relay-app>`, `<normalizer-app>`, `<authz-app>` por los nombres reales de las Function Apps del ambiente (Managed Identity System-Assigned = nombre de la app).

4.0 Prerequisito: esquema y tablas desplegadas (DB project / scripts)

Antes de Change Tracking, grants o cursores, las tablas tienen que EXISTIR. La DDL de las tablas y triggers vive en el repo de base de datos (`VieCloud_DB_Colombia_Schema`) y la ejecuta el DBA. Estas son las tablas que tienen que quedar creadas:

Base / Schema	Tablas
Tenant — schemas Outbox	<code>Clinical.OutboxEvent</code> , <code>Hospitalization.OutboxEvent</code> , <code>Billing.OutboxEvent</code> , <code>Admissions.OutboxEvent</code>

Base / Schema	Tablas
Tenant — schema <code>Authorization</code>	<code>AuthorizationControl</code> , <code>AuthorizationControlTrace</code> , <code>ProcessedInbox</code>
Tenant — schema <code>Authorization</code> (agente)	<code>AdmissionAuthorizerAssignmentLog</code> , <code>UsersAssignment</code> , <code>UserNovelties</code> , <code>AuthorizationManagementParameters</code>
Control Plane — schema <code>Platform</code>	<code>TenantCatalog</code> , <code>TenantOutboxCursor</code> , <code>TenantOutboxLease</code> , <code>PoolThrottlingConfig</code>

Más los **triggers** e **índice**: - `TR_ADINGRESO_AdmissionStatusChanged` sobre `dbo.ADINGRESO` — emite el evento de admisión al outbox (solo con `TIPOINGRE=2`). - Trigger de **backfill**

`AdmissionAuthorizerAssignmentLog` → `AuthorizationControl.AssignedAuthorizer`. - Índice único filtrado de `AdmissionAuthorizerAssignmentLog` (`WHERE IsCurrent=1`) — "una asignación vigente por ingreso".

De dónde sale cada cosa:

Origen	Qué aporta
DACPAC del DB project (<code>VieCloud_DB_Colombia_Schema</code>)	Crea/actualiza todas las tablas y triggers — incluidas las del agente (<code>Admissions.OutboxEvent</code> , las 4 tablas de <code>Authorization</code> , y los 3 triggers). Es la fuente de verdad de la DDL (reproducida abajo como referencia).
<code>Vie_Erp/Authorization/Scripts/ERP84_OnboardingRunbook.sql</code>	PARTE 1 (Control Plane): tablas <code>Platform</code> + registra el tenant + siembra cursores + usuario/grants del Relay. PARTE 2 (tenant): Change Tracking (§4.2) + usuarios/grants de las 3 Managed Identities (§4.3).
<code>Deploy_BD636_DistribucionAutorizadores.sql</code>	Grants + Change Tracking específicos del agente por ambiente/tenant (<code>VIEW CHANGE TRACKING</code> sobre <code>Admissions.OutboxEvent</code> , grants del Authz sobre las tablas del agente).

Las tablas outbox base (`Clinical / Hospitalization / Billing.OutboxEvent`), `dbo.ADINGRESO` y el schema `Authorization` base ya existen en un tenant productivo. Lo **nuevo del agente** es lo que hay que sumar (tablas + triggers del DACPAC + grants/CT del script del agente).

DDL de las tablas del Control Plane (`Platform`) — se ejecuta en la base del Control Plane (ej. `INDIGOSEC`). Es la PARTE 1.1 del runbook, reproducida aquí como referencia:

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'Platform')
    EXEC('CREATE SCHEMA [Platform] AUTHORIZATION [dbo];');
GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE SCHEMA_NAME(schema_id)='Platform' AND
name='TenantCatalog')
CREATE TABLE [Platform].[TenantCatalog] (
    [TenantId]          UNIQUEIDENTIFIER NOT NULL,
    [TenantCode]        NVARCHAR(50)     NOT NULL,
    [DatabaseName]      NVARCHAR(128)    NOT NULL,
    [ServerName]        NVARCHAR(255)    NOT NULL,
    [ElasticPoolName]   NVARCHAR(128)    NULL,
    [Region]            NVARCHAR(50)     NOT NULL,
```

```

[IsActive]          BIT          DEFAULT ((1)) NOT NULL,
[Status]            NVARCHAR(50)  DEFAULT ('ACTIVE') NOT NULL,
[CreatedAtUtc]      DATETIME2(7)  NOT NULL,
[UpdatedAtUtc]      DATETIME2(7)  NOT NULL,
CONSTRAINT [PK_TenantCatalog] PRIMARY KEY CLUSTERED ([TenantId] ASC),
CONSTRAINT [UQ_TenantCatalog_Code] UNIQUE NONCLUSTERED ([TenantCode] ASC)
);
GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE SCHEMA_NAME(schema_id)='Platform' AND
name='TenantOutboxCursor')
CREATE TABLE [Platform].[TenantOutboxCursor] (
    [TenantId]          UNIQUEIDENTIFIER NOT NULL,
    [OutboxName]        NVARCHAR(100)  DEFAULT ('Clinical.OutboxEvent') NOT NULL,
    [LastSyncVersion]   BIGINT          DEFAULT ((-1)) NOT NULL,
    [LastPollAtUtc]     DATETIME2(7)    NULL,
    [LastSuccessAtUtc]  DATETIME2(7)    NULL,
    [ErrorCount]        INT             DEFAULT ((0)) NOT NULL,
    [UpdatedAtUtc]      DATETIME2(7)    NOT NULL,
    CONSTRAINT [PK_TenantOutboxCursor] PRIMARY KEY CLUSTERED ([TenantId] ASC, [OutboxName] ASC)
);
GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE SCHEMA_NAME(schema_id)='Platform' AND
name='TenantOutboxLease')
CREATE TABLE [Platform].[TenantOutboxLease] (
    [TenantId]          UNIQUEIDENTIFIER NOT NULL,
    [LeaseOwner]        NVARCHAR(255)   NOT NULL,
    [LeaseExpiresAtUtc] DATETIME2(7)    NOT NULL,
    [AcquiredAtUtc]     DATETIME2(7)    NOT NULL,
    CONSTRAINT [PK_TenantOutboxLease] PRIMARY KEY CLUSTERED ([TenantId] ASC)
);
GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE SCHEMA_NAME(schema_id)='Platform' AND
name='PoolThrottlingConfig')
CREATE TABLE [Platform].[PoolThrottlingConfig] (
    [ElasticPoolName]   NVARCHAR(128) NOT NULL,
    [MaxConcurrentTenants] INT         DEFAULT ((5)) NOT NULL,
    [BatchSizePerTenant] INT          DEFAULT ((100)) NOT NULL,
    [PollIntervalActiveMs] INT        DEFAULT ((5000)) NOT NULL,
    [PollIntervalIdleMs] INT          DEFAULT ((30000)) NOT NULL,
    [UpdatedAtUtc]      DATETIME2(7)  NOT NULL,
    CONSTRAINT [PK_PoolThrottlingConfig] PRIMARY KEY CLUSTERED ([ElasticPoolName] ASC)
);
GO

```

DDL de las tablas Outbox (base del tenant). Las **4** ([Clinical](#) , [Hospitalization](#) , [Billing](#) , [Admissions](#)) tienen el **MISMO contrato**: log append-only, [BusinessEventHash](#) calculado (dedup de idempotencia), [PayloadHash](#) calculado, PK por [OutboxId](#) . Solo cambian el **schema** y los **nombres**

de constraints. Plantilla (reemplazar `{Schema}` y los sufijos de constraint según la tabla de abajo):

```
CREATE TABLE [{Schema}].[OutboxEvent]
(
    [OutboxId]          BIGINT IDENTITY (1, 1) NOT NULL,
    [EventId]           UNIQUEIDENTIFIER CONSTRAINT [DF_{Sfx}OutboxEvent_EventId] DEFAULT
(NEWID()) NOT NULL,
    [EventType]         NVARCHAR(100) NOT NULL,          -- ej. clinical.record-committed.v1 /
billing.stay-committed.v1 / clinical.admission-registered.v1
    [AggregateType]     NVARCHAR(100) NOT NULL,
    [AggregateId]       NVARCHAR(255) NOT NULL,
    [BusinessEventHash] AS (HASHBYTES('SHA2_256', CONCAT([EventType], '|',
[AggregateId], '|', CONVERT(NVARCHAR(30), [OccurredAtUtc], 126)))) PERSISTED NOT NULL,
    [PayloadJson]       NVARCHAR(MAX) NOT NULL,
    [PayloadHash]       AS (HASHBYTES('SHA2_256', CONVERT(VARBINARY(MAX), [PayloadJson])))
PERSISTED NOT NULL,
    [OccurredAtUtc]     DATETIME2(7) NOT NULL,
    [CreatedAtUtc]      DATETIME2(7) CONSTRAINT [DF_{Sfx}OutboxEvent_CreatedAtUtc] DEFAULT
(SYSUTCDATETIME()) NOT NULL,
    CONSTRAINT [PK_{Sfx}OutboxEvent] PRIMARY KEY CLUSTERED ([OutboxId] ASC),
    CONSTRAINT [UQ_{Sfx}OutboxEvent_BusinessEventHash] UNIQUE NONCLUSTERED ([BusinessEventHash]
ASC) -- idempotencia
);
GO
```

Nombres reales por tabla (`{Schema}` / `{Sfx}` en los constraints):

Tabla	{Schema}	Sufijo de constraint {Sfx}
Clinical.OutboxEvent	Clinical	(vacío) → DF_OutboxEvent_EventId , PK_OutboxEvent , UQ_OutboxEvent_BusinessEventHash
Hospitalization.OutboxEvent	Hospitalization	Hosp → DF_HospOutboxEvent_EventId , PK_HospOutboxEvent , UQ_HospOutboxEvent_BusinessEventHash
Billing.OutboxEvent	Billing	Billing → DF_BillingOutboxEvent_EventId , PK_BillingOutboxEvent , UQ_BillingOutboxEvent_BusinessEventHash
Admissions.OutboxEvent	Admissions	Admissions → DF_AdmissionsOutboxEvent_EventId , PK_AdmissionsOutboxEvent , UQ_AdmissionsOutboxEvent_BusinessEventHash

Cada schema (`Clinical` , `Hospitalization` , `Billing` , `Admissions`) debe existir antes (`CREATE SCHEMA [X] AUTHORIZATION [dbo];`). En un tenant productivo `Clinical` / `Hospitalization` / `Billing` ya están; la nueva es `Admissions` (agente).

DDL de las tablas del Agente (base del tenant, schema `Authorization`):

```
CREATE TABLE [Authorization].[UsersAssignment] (
    [Id]          INT IDENTITY(1,1) NOT NULL,
```

```

    [UserCode] VARCHAR(20) NOT NULL,          -- usuario de seguridad (INDIGOSEC2),
referencia lógica
    [FullName] NVARCHAR(100) NOT NULL,
    [Status] BIT NOT NULL CONSTRAINT [DF_UsersAssignment_Status] DEFAULT ((1)), --
1=activo
    [EntryType] TINYINT NOT NULL CONSTRAINT [DF_UsersAssignment_EntryType] DEFAULT ((0)), --
1=Hospitalario
    [IsRemoved] BIT NOT NULL CONSTRAINT [DF_UsersAssignment_IsRemoved] DEFAULT ((0)),
    CONSTRAINT [PK_UsersAssignment] PRIMARY KEY CLUSTERED ([Id] ASC)
);
GO
CREATE NONCLUSTERED INDEX [IX_UsersAssignment_UserCode] ON [Authorization].[UsersAssignment]
([UserCode]) WHERE [IsRemoved] = 0;
GO

CREATE TABLE [Authorization].[UserNovelties] (
    [Id] INT IDENTITY(1,1) NOT NULL,
    [AssignedUserId] INT NOT NULL,
    [NoveltyDate] DATETIME NOT NULL CONSTRAINT [DF_UserNovelties_NoveltyDate] DEFAULT
(GETUTCDATE()),
    [Description] NVARCHAR(300) NOT NULL,
    [IsUserActive] BIT NOT NULL CONSTRAINT [DF_UserNovelties_IsUserActive] DEFAULT ((0)), -- 0
= inactivo por la novedad
    [TypeNovely] TINYINT NULL,
    [EndDate] DATETIME NULL,
    CONSTRAINT [PK_UserNovelties] PRIMARY KEY CLUSTERED ([Id] ASC),
    CONSTRAINT [FK_UserNovelties_UsersAssignment] FOREIGN KEY ([AssignedUserId]) REFERENCES
[Authorization].[UsersAssignment] ([Id])
);
GO
CREATE NONCLUSTERED INDEX [IX_UserNovelties_AssignedUserId] ON [Authorization].[UserNovelties]
([AssignedUserId]) INCLUDE ([NoveltyDate]);
GO

CREATE TABLE [Authorization].[AuthorizationManagementParameters] (
    [Id] INT IDENTITY(1,1) NOT NULL,
    [AutomaticAssignment] BIT NOT NULL,          -- interruptor global
    [EntryType] VARCHAR(50) NULL,              -- tipos de ingreso, separados por coma
(ej. "1")
    [StartDateAssignment] DATETIME NULL,
    [CreationUser] VARCHAR(20) NOT NULL,
    [CreationDate] DATETIME NOT NULL CONSTRAINT
[DF_AuthorizationManagementParameters_CreationDate] DEFAULT (GETUTCDATE()),
    [ModificationUser] VARCHAR(20) NULL,
    [ModificationDate] DATETIME NULL,
    CONSTRAINT [PK_AuthorizationManagementParameters] PRIMARY KEY CLUSTERED ([Id] ASC)
);
GO

CREATE TABLE [Authorization].[AdmissionAuthorizerAssignmentLog] (

```

```

    [Id] INT IDENTITY(1,1) NOT NULL,
    [AdmissionNumber] CHAR(10) NOT NULL,
    [AssignedUserCode] VARCHAR(20) NOT NULL,
    [PreviousAssignedUserCode] VARCHAR(20) NULL,
    [Action] VARCHAR(20) NOT NULL CONSTRAINT
[DF_AdmissionAuthorizerAssignmentLog_Action] DEFAULT (N'Automatica'),
    [IsCurrent] BIT NOT NULL CONSTRAINT
[DF_AdmissionAuthorizerAssignmentLog_IsCurrent] DEFAULT ((1)),
    [AssignmentDate] DATETIME NOT NULL CONSTRAINT
[DF_AdmissionAuthorizerAssignmentLog_AssignmentDate] DEFAULT (GETUTCDATE()),
    CONSTRAINT [PK_AdmissionAuthorizerAssignmentLog] PRIMARY KEY CLUSTERED ([Id] ASC)
);
GO
-- A lo sumo UNA asignación vigente por ingreso (= guarda de idempotencia de negocio)
CREATE UNIQUE NONCLUSTERED INDEX [UX_AdmissionAuthorizerAssignmentLog_AdmissionNumber_Current]
    ON [Authorization].[AdmissionAuthorizerAssignmentLog] ([AdmissionNumber]) WHERE [IsCurrent] =
1;
GO
-- Soporta el COUNT por usuario del algoritmo de "menor carga"
CREATE NONCLUSTERED INDEX [IX_AdmissionAuthorizerAssignmentLog_UserCode_Current]
    ON [Authorization].[AdmissionAuthorizerAssignmentLog] ([AssignedUserCode]) WHERE [IsCurrent]
= 1;
GO

```

Trigger 1 — `dbo.ADINGRESO` → emite el evento de admisión al outbox (solo Hospitalario, `TIPOINGRE=2`):

```

CREATE TRIGGER [dbo].[TR_ADINGRESO_AdmissionStatusChanged]
    ON [dbo].[ADINGRESO]
    AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        -- ALTA: fila nueva creada como Hospitalario
        INSERT INTO [Admissions].[OutboxEvent] ([EventType],[AggregateType],[AggregateId],
[PayloadJson],[OccurredAtUtc])
        SELECT 'clinical.admission-registered.v1', 'Admission', i.[NUMINGRES],
        (SELECT i.[NUMINGRES] AS admissionNumber, i.[IPCODPACI] AS patientCode, i.[IFECHAING]
AS admissionDate,
        i.[IESTADOIN] AS admissionStatus, i.[CODENTIDA] AS entityCode, i.[CODCENATE]
AS careCenterCode
        FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
        SYSUTCDATETIME()
        FROM inserted i
        WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.[NUMINGRES] = i.[NUMINGRES])
        AND i.[TIPOINGRE] = 2;

        -- CAMBIO DE ESTADO: fila existente cuyo IESTADOIN cambió
        IF UPDATE(IESTADOIN)
    
```

```

BEGIN
    INSERT INTO [Admissions].[OutboxEvent] ([EventType],[AggregateType],[AggregateId],
[PayloadJson],[OccurredAtUtc])
        SELECT 'clinical.admission-status-changed.v1', 'Admission', i.[NUMINGRES],
            (SELECT i.[NUMINGRES] AS admissionNumber, i.[IPCODPACI] AS patientCode, i.
[IFECHAING] AS admissionDate,
                i.[IESTADOIN] AS admissionStatus, i.[CODENTIDA] AS entityCode, i.
[CODCENATE] AS careCenterCode
            FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
        SYSUTCDATETIME()
    FROM inserted i
    INNER JOIN deleted d ON d.[NUMINGRES] = i.[NUMINGRES]
    WHERE i.[IESTADOIN] <> d.[IESTADOIN] AND i.[TIPOINGRE] = 2;
END
END TRY
BEGIN CATCH
    IF ERROR_NUMBER() NOT IN (2627, 2601) THROW; -- choque UNIQUE(BusinessEventHash) =
evento ya emitido, no es error
END CATCH
END
GO

```

Nota de diseño (limitación conocida): este trigger dispara la rama de cambio solo con `IF UPDATE(IESTADOIN)` y `IESTADOIN <> anterior`. La **asignación de cama** (hospitalizar) cambia `TIPOINGRE` a 2 pero **no** `IESTADOIN`, así que por esa vía NO se emite el evento — hay que emitirlo desde el productor (`IND_CHO_Transaccion_AsignarCamas` / `IND_CHO_GrabarReingresoCamas` en `EHR_ServicesCore`, vía `AdmissionsOutboxCommand`) o ampliar el trigger para detectar la transición `TIPOINGRE → 2`.

Trigger 2 — backfill `AdmissionAuthorizerAssignmentLog` →

`AuthorizationControl.AssignedAuthorizer` (cuando el ingreso se asigna DESPUÉS de que ya existían controles sin asignar):

```

CREATE TRIGGER [Authorization].[TR_AdmissionAuthorizerAssignmentLog_BackfillAuthorizationControl]
    ON [Authorization].[AdmissionAuthorizerAssignmentLog]
    AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE ac SET ac.[AssignedAuthorizer] = i.[AssignedUserCode]
    FROM [Authorization].[AuthorizationControl] ac
    INNER JOIN inserted i ON i.[AdmissionNumber] = ac.[AdmissionNumber]
    WHERE i.[IsCurrent] = 1 AND ac.[AssignedAuthorizer] IS NULL; -- nunca pisa una asignación
manual
END
GO

```

Trigger 3 — simétrico, en `AuthorizationControl` (cuando el control se crea DESPUÉS de que el ingreso ya estaba asignado):

```

CREATE TRIGGER [Authorization].[TR_AuthorizationControl_PropagateAdmissionAssignment]
    ON [Authorization].[AuthorizationControl]
    AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE ac SET ac.[AssignedAuthorizer] = aal.[AssignedUserCode]
    FROM [Authorization].[AuthorizationControl] ac
    INNER JOIN inserted i ON i.[Id] = ac.[Id]
    INNER JOIN [Authorization].[AdmissionAuthorizerAssignmentLog] aal
        ON aal.[AdmissionNumber] = i.[AdmissionNumber] AND aal.[IsCurrent] = 1
    WHERE ac.[AssignedAuthorizer] IS NULL;
END
GO

```

Los dos triggers de backfill son **complementarios**: uno cubre "el ingreso se asigna después del control", el otro "el control se crea después de la asignación". Juntos garantizan que `AuthorizationControl.AssignedAuthorizer` quede poblado sin importar el orden de llegada.

Verificar existencia antes de seguir:

```

SELECT s.name AS [schema], t.name AS [table]
FROM sys.tables t JOIN sys.schemas s ON s.schema_id=t.schema_id
WHERE (s.name='Admissions' AND t.name='OutboxEvent')
    OR (s.name='Authorization' AND t.name IN
('AdmissionAuthorizerAssignmentLog','UsersAssignment','UserNovelties','AuthorizationManagementParameters'))
    OR (s.name='Platform');

```

4.1 Service Bus

Hay que crear los **topics** (canales) y sus **suscripciones** (buzones de cada consumidor). Un topic reparte una copia de cada mensaje a **todas** sus suscripciones (fan-out). Cada suscripción lleva un **filtro** (`SqlFilter`) para que solo reciba lo que le corresponde.

Topics y suscripciones a crear:

Topic	Suscripción	Consumidor	Filtro (SqlFilter)
ehr-source-events	orders-normalizer	Normalizer (órdenes/urgencias/ingresos)	NOT (eventType LIKE 'rda.%) AND NOT (eventType LIKE 'billing.%')
ehr-source-events	billing-stay	BillingStayNormalizer (estancias)	eventType LIKE 'billing.%'
clinical-order-facts	authorization	AuthorizationOrderConsumer	eventType = 'clinical.order-normalized.v1' AND schemaVersion = '1'
clinical-emergency-facts	authorization	AuthorizationEmergencyInitialCareConsumer	eventType = 'clinical-emergency-normalized.v1'

Topic	Suscripción	Consumidor	Filtro (SqlFilter)
clinical-stay-facts	authorization	AuthorizationStayConsumer	eventType = 'clinical-stay-control.v1'
clinical-admission-facts	authorization	AuthorizationAdmissionDistributionConsumer (agente)	eventType = 'clinical-admission-normalized.v1'
relay-dispatch (cola, no topic)	—	interno del Relay	—

Cómo crearlos (Azure CLI). Reemplazar `<rg-sb>` = RG del namespace y `<ns>` = nombre del namespace (ej. `global-multi-fhir-dev`). Repetir por cada fila de la tabla.

```
# 1) Crear el TOPIC (con deduplicación) – uno por cada topic de la tabla
az servicebus topic create \
  --resource-group <rg-sb> --namespace-name <ns> \
  --name clinical-admission-facts --enable-duplicate-detection true

# 2) Crear la SUSCRIPCIÓN dentro del topic – con DLQ y max delivery 10
az servicebus topic subscription create \
  --resource-group <rg-sb> --namespace-name <ns> \
  --topic-name clinical-admission-facts --name authorization \
  --max-delivery-count 10 --dead-lettering-on-message-expiration true

# 3) Poner el FILTRO: se borra la regla por defecto ($Default = 1=1) y se crea una con el
SqlFilter

az servicebus topic subscription rule delete \
  --resource-group <rg-sb> --namespace-name <ns> \
  --topic-name clinical-admission-facts --subscription-name authorization --name '$Default'

az servicebus topic subscription rule create \
  --resource-group <rg-sb> --namespace-name <ns> \
  --topic-name clinical-admission-facts --subscription-name authorization \
  --name contract-filter \
  --filter-sql-expression "eventType = 'clinical-admission-normalized.v1'"
```

También se pueden crear desde el **portal**: Namespace → Topics → + *Topic* (activar *Duplicate detection*) → dentro del topic, Subscriptions → + *Subscription* (Max delivery count 10, *Enable dead-lettering on message expiration*) → Filters → editar la regla `$Default` y reemplazar `1=1` por el SqlFilter de la tabla.

RBAC (Managed Identity → rol scoped al topic/cola, NO al namespace):

Managed Identity	Rol	Alcance
<code><relay-app></code>	Azure Service Bus Data Sender	topic <code>ehr-source-events</code> + cola <code>relay-dispatch</code> (Sender/Receiver)
	Data Receiver	<code>ehr-source-events</code>

Managed Identity	Rol	Alcance
<normalizer-app>		
<normalizer-app>	Data Sender	clinical-order-facts , clinical-emergency-facts , clinical-stay-facts , clinical-admission-facts
<authz-app>	Data Receiver	clinical-order-facts , clinical-emergency-facts , clinical-stay-facts , clinical-admission-facts

Cómo asignar el RBAC (Azure CLI). El rol se asigna **scoped al topic/cola** (no al namespace). Como la MI vive en *VIE Development* y el namespace en *Indira Development*, se asigna en la suscripción del **namespace** usando el `principalId` (Object ID) de la MI.

```
# 0) Obtener el principalId (Object ID) de la MI de cada Function App (en VIE Development)
az functionapp identity show \
  --resource-group rg-erp84-clinical-dev --name ehrco-normalizer-dev-erp84d \
  --query principalId -o tsv

# 1) Asignar el rol scoped a UN topic (repetir por cada fila de la tabla de arriba).
#   --role: "Azure Service Bus Data Sender" o "Azure Service Bus Data Receiver".
#   --scope: el resourceId del topic (o cola) en el namespace de Service Bus (Indira Development).
az role assignment create \
  --assignee <principalId-de-la-MI> \
  --role "Azure Service Bus Data Sender" \
  --scope "/subscriptions/<sub-id-indira-dev>/resourceGroups/global-coral-interoperability-service/providers/Microsoft.ServiceBus/namespaces/global-multi-fhir-dev/topics/clinical-admission-facts"

# Ejemplo para la cola interna del Relay (relay-dispatch): Sender + Receiver
az role assignment create --assignee <relay-principalId> --role "Azure Service Bus Data Sender" \
  --scope ".../namespaces/global-multi-fhir-dev/queues/relay-dispatch"
az role assignment create --assignee <relay-principalId> --role "Azure Service Bus Data Receiver" \
  --scope ".../namespaces/global-multi-fhir-dev/queues/relay-dispatch"
```

Quien corra estos comandos necesita permiso para **crear role assignments en la suscripción/RG de Indira Development** (rol *Role Based Access Control Administrator* o *User Access Administrator* sobre ese RG). Es el paso cross-subscription que hay que coordinar con el dueño del namespace.

Gap conocido (agente): tras redeploy del Normalizer, faltaba `Data Sender` sobre `clinical-admission-facts` → los admission facts no se publicaban. Verificar este rol explícitamente.

RBAC de Cosmos (data-plane, NO es el RBAC de Azure normal). El Normalizer **escribe** el documento claim-check y el Authz lo **lee**. Asignar el rol data-plane de Cosmos (`sqlRoleAssignments`), scoped a la base/container `NormalizedMedicalOrders` :

Managed Identity	Rol Cosmos	Motivo
<normalizer-app>	Cosmos DB Built-in Data Contributor (00000000-0000-0000-0000-000000000002)	upsert del documento

Managed Identity	Rol Cosmos	Motivo
<authz-app>	Cosmos DB Built-in Data Reader (00000000-0000-0000-0000-000000000001)	lectura del claim-check

```
# Ejemplo (repetir para cada MI con su principalId y su roleDefinitionId):
az cosmosdb sql role assignment create \
  --resource-group <rg-cosmos> --account-name <cosmos-account> \
  --role-definition-id 00000000-0000-0000-0000-000000000002 \
  --principal-id <normalizer-principalId> \
  --scope "/dbs/<database>/colls/NormalizedMedicalOrders"
```

Es rol **data-plane** de Cosmos (control de acceso a los datos), distinto del RBAC de Azure (control plane). Sin él, el Normalizer falla al hacer [Upsert](#) y el Authz no puede leer el claim-check (display clínico queda NULL).

4.2 Change Tracking (base del tenant)

```
-- A nivel de base
ALTER DATABASE CURRENT SET CHANGE_TRACKING = ON (CHANGE_RETENTION = 7 DAYS, AUTO_CLEANUP = ON);
ALTER DATABASE CURRENT SET ALLOW_SNAPSHOT_ISOLATION ON;

-- Por cada tabla outbox que drena el Relay (OJO: CHANGE_TRACKING es UNA palabra con guion bajo)
ALTER TABLE [Clinical].[OutboxEvent]          ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
ALTER TABLE [Hospitalization].[OutboxEvent]    ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
ALTER TABLE [Billing].[OutboxEvent]            ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
ALTER TABLE [Admissions].[OutboxEvent]        ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
-- agente
```

4.3 Grants SQL por Managed Identity (EL PUNTO CRÍTICO)

Cada Function App conecta con su Managed Identity y necesita sus permisos en la **base del tenant**. [GRANT SELECT](#) NO cubre [EXECUTE](#) de funciones ni [VIEW CHANGE TRACKING](#).

Relay — solo outbox + change tracking:

```
CREATE USER [<relay-app>] FROM EXTERNAL PROVIDER; -- si no existe
GRANT SELECT          ON [Clinical].[OutboxEvent]          TO [<relay-app>];
GRANT VIEW CHANGE TRACKING ON [Clinical].[OutboxEvent]      TO [<relay-app>];
GRANT SELECT          ON [Hospitalization].[OutboxEvent]    TO [<relay-app>];
GRANT VIEW CHANGE TRACKING ON [Hospitalization].[OutboxEvent] TO [<relay-app>];
GRANT SELECT          ON [Billing].[OutboxEvent]            TO [<relay-app>];
GRANT VIEW CHANGE TRACKING ON [Billing].[OutboxEvent]        TO [<relay-app>];
-- Agente:
GRANT SELECT          ON [Admissions].[OutboxEvent]         TO [<relay-app>];
GRANT VIEW CHANGE TRACKING ON [Admissions].[OutboxEvent]    TO [<relay-app>];
```

Gap conocido (agente): sin `VIEW CHANGE TRACKING` sobre `Admissions.OutboxEvent`, `CHANGE_TRACKING_MIN_VALID_VERSION()` devuelve `NULL` **en silencio** → el tenant entra en loop `REQUIRES_RESYNC` y nunca publica. No lanza error.

Normalizer — lectura clínica + contexto + estancias:

```
CREATE USER [<normalizer-app>] FROM EXTERNAL PROVIDER;
GRANT SELECT ON SCHEMA::dbo TO [<normalizer-app>];
GRANT SELECT ON SCHEMA::Contract TO [<normalizer-app>];
GRANT SELECT ON SCHEMA::Clinical TO [<normalizer-app>];
GRANT SELECT ON SCHEMA::Billing TO [<normalizer-app>]; -- estancias
(BillingStayControlReader)
GRANT EXECUTE ON OBJECT::dbo.PatientDestiny TO [<normalizer-app>]; -- función escalar de los
readers de estancia
GRANT EXECUTE ON OBJECT::Common.GETDATE TO [<normalizer-app>]; -- (agente) usado por
lectura de admisión
```

AuthorizationControl — susceptibilidad + escritura del control + (agente) asignación:

```
CREATE USER [<authz-app>] FROM EXTERNAL PROVIDER;
GRANT SELECT ON SCHEMA::dbo TO [<authz-app>]; -- ADCONFSE*/ADINGRESO/INPACIENT/
maestros
GRANT SELECT ON SCHEMA::Contract TO [<authz-app>];
GRANT SELECT ON SCHEMA::Inventory TO [<authz-app>];
GRANT SELECT ON SCHEMA::Clinical TO [<authz-app>];
GRANT SELECT ON SCHEMA::Authorization TO [<authz-app>]; -- lee AuthorizationControl (dedup),
ProcessedInbox y tablas del agente
GRANT EXECUTE ON OBJECT::Common.GETDATE TO [<authz-app>]; -- DEFAULT Common.GETDATE() en
AuthorizationControl.CreationDate y AuthorizationControlTrace.ActionDate
GRANT INSERT, UPDATE ON [Authorization].[AuthorizationControl] TO [<authz-app>];
GRANT INSERT, UPDATE ON [Authorization].[AuthorizationControlTrace] TO [<authz-app>];
GRANT SELECT, INSERT, UPDATE ON [Authorization].[ProcessedInbox] TO [<authz-app>];
-- Agente:
GRANT INSERT, SELECT, UPDATE ON [Authorization].[AdmissionAuthorizerAssignmentLog] TO [<authz-
app>];
GRANT SELECT ON [Authorization].[UsersAssignment] TO [<authz-app>];
GRANT SELECT ON [Authorization].[UserNovelties] TO [<authz-app>];
GRANT SELECT ON [Authorization].[AuthorizationManagementParameters] TO [<authz-app>];
-- Estancias – alerta "estancia no liquidada por Stella":
GRANT INSERT, UPDATE ON [Authorization].[AuthorizationControlAlert] TO [<authz-app>];
```

Gap histórico: el grant original del Authz solo daba `SCHEMA::Clinical` y fallaba tabla por tabla (`dbo.ADINGRESO` permission denied). Debe cubrir `dbo/Contract/Inventory/Authorization/Clinical`.
Estancias desde HC (2026-08): la orden de traslado del médico (`HCHISPACA.INDICAPAC` ∈ {3,4,5,6,9,21}) crea/actualiza la estancia (SubjectType 5) reutilizando `clinical.record-committed.v1` → `clinical-stay-facts`; **no requiere objetos de BD nuevos**. La alerta de "estancia no liquidada" inserta en `Authorization.AuthorizationControlAlert` (tabla ya existente en el DDL) — por eso el grant `INSERT, UPDATE` de arriba. La MI del Authz debe tenerlo o la alerta falla con `INSERT permission denied on 'AuthorizationControlAlert'` (Error 229).

4.4 Control Plane (registro del tenant)

En la base con schema `Platform`:

```
-- 1. Registrar el tenant (o verificar IsActive=1 AND Status='ACTIVE')
INSERT INTO [Platform].[TenantCatalog] (TenantId, TenantCode, DatabaseName, ServerName, Region,
IsActive, Status, CreatedAtUtc, UpdatedAtUtc)
VALUES (NEWID(), '<codigo>', '<DatabaseName>', '<ServerName>', '<region>', 1, 'ACTIVE',
SYSUTCDATETIME(), SYSUTCDATETIME());

-- 2. Sembrar el cursor por cada outbox (incluye Admissions.OutboxEvent para el agente)
INSERT INTO [Platform].[TenantOutboxCursor] (TenantId, OutboxName, LastSyncVersion, ErrorCount,
UpdatedAtUtc)
SELECT tc.TenantId, v.OutboxName, -1, 0, SYSUTCDATETIME()
FROM [Platform].[TenantCatalog] tc
CROSS JOIN (VALUES ('Clinical.OutboxEvent'),('Hospitalization.OutboxEvent'),
('Billing.OutboxEvent'),('Admissions.OutboxEvent')) AS v(OutboxName)
WHERE tc.Status='ACTIVE' AND NOT EXISTS (SELECT 1 FROM [Platform].[TenantOutboxCursor] c WHERE
c.TenantId=tc.TenantId AND c.OutboxName=v.OutboxName);

-- 3. Grants sobre Platform.* (Control Plane). El Relay lee y avanza cursores/leases;
-- Normalizer y Authz solo resuelven el tenant (leen TenantCatalog).
GRANT SELECT ON [Platform].[TenantCatalog] TO [<relay-app>];
GRANT SELECT, INSERT, UPDATE ON [Platform].[TenantOutboxCursor] TO [<relay-app>];
GRANT SELECT, INSERT, UPDATE, DELETE ON [Platform].[TenantOutboxLease] TO [<relay-app>];
GRANT SELECT ON [Platform].[PoolThrottlingConfig] TO [<relay-app>];

GRANT SELECT ON [Platform].[TenantCatalog] TO [<normalizer-app>];
GRANT SELECT ON [Platform].[TenantCatalog] TO [<authz-app>];
```

Los usuarios `<normalizer-app>` / `<authz-app>` deben existir en el Control Plane (`CREATE USER ... FROM EXTERNAL PROVIDER`) para poder darles el grant. El **TenantId** lo define el catálogo (`NEWID()`). El Relay taguea cada evento con ese GUID; el Normalizer/Authz lo resuelven contra el catálogo. Un evento cuyo tenant no esté ACTIVE en el catálogo → `TenantNotActiveException` (el código lo ignora y completa el mensaje).

4.5 App settings (Function Apps)

Vía App Settings + Key Vault (`@Microsoft.KeyVault(...)`), autenticando con Managed Identity:

Setting	App(s)	Valor
<code>ControlPlane:ConnectionString</code>	relay, normalizer, authz	cadena al Platform DB (<code>Authentication=Active Directory Managed Identity</code>)
<code>Tenant:ConnectionStringTemplate</code>	relay, normalizer, authz	plantilla <code>{ServerName} / {DatabaseName}</code> (MI)
<code>EhrSourceEventsConnection__fullyQualifiedNamespace</code>	relay, normalizer	<code><ns>.servicebus.windows.net</code>
<code>ClinicalOrderFactsConnection__fullyQualifiedNamespace</code>	authz	<code><ns>.servicebus.windows.net</code>

Setting	App(s)	Valor
Cosmos:AccountEndpoint / Cosmos:DatabaseId	normalizer	endpoint + base
Relay:OutboxTables	relay	agregar <code>Admissions.OutboxEvent</code> (opt-in; NO está en el default) para el agente
Authorization:SystemUser	authz	usuario de sistema

4.6 Datos semilla del Agente Autorizador (sin esto NO asigna)

```
-- Interruptor global de asignación automática
-- (si falta o AutomaticAssignment=0 → outcome 'AutomaticAssignmentDisabled')
SELECT * FROM [Authorization].[AuthorizationManagementParameters]; -- AutomaticAssignment = 1

-- Pool de autorizadores (si vacío → 'NoEligibleUsers', queda para asignación manual)
SELECT * FROM [Authorization].[UsersAssignment]
WHERE Status=1 AND IsRemoved=0 AND EntryType=1; -- EntryType 1 = Hospitalario
```

5. Verificación end-to-end

- Relay drenando:** `SELECT * FROM [Platform].[TenantOutboxCursor] → LastSuccessAtUtc` reciente y `LastSyncVersion` avanzando por cada outbox. Sin `REQUIRES_RESYNC`.
- Normalizer:** logs `Evento recibido... → HC leída Órdenes=N → Upsert Cosmos OK → Publicado ...`. Para admisión: `Evento de ingreso normalizado OK`.
- AuthorizationControl (órdenes/urgencias/estancias):** `sql SELECT Id, SubjectType, AdmissionNumber, SourceTable, RequestedQuantity, Estado, FechaCreacion FROM [Authorization].[AuthorizationControl] ORDER BY Id DESC;`
- Agente autorizador:** `sql SELECT * FROM [Authorization].[AdmissionAuthorizerAssignmentLog] ORDER BY AssignmentDate DESC; -- Action='Automatica', IsCurrent=1 SELECT NumeroIngreso, AssignedAuthorizer FROM [Authorization].[AuthorizationControl] WHERE AssignedAuthorizer IS NOT NULL;`

6. Troubleshooting (síntoma → causa → fix)

Síntoma	Causa raíz	Fix
Tenant en loop <code>REQUIRES_RESYNC</code> , Relay nunca publica	Falta <code>VIEW CHANGE TRACKING</code> sobre la tabla outbox → <code>MIN_VALID_VERSION</code> = NULL silencioso	<code>GRANT VIEW CHANGE TRACKING ON [Schema].[Table] TO [User]</code> (SELECT) a la MI del R (\$4.3)
<code>EXECUTE permission denied on 'PatientDestiny'</code> (error 229)	Reader de estancia usa función escalar; <code>SELECT</code> no cubre <code>EXECUTE</code>	<code>GRANT EXECUTE ON [Schema].[Function] TO [User]</code> a la MI del Normalizer
<code>SELECT permission denied on 'AccountControlStays'</code> (Billing)	Normalizer sin <code>SELECT</code> sobre schema <code>Billing</code>	<code>GRANT SELECT ON [Schema]::[Table] TO [User]</code> a la MI del Normalizer

Síntoma	Causa raíz	Fix
EXECUTE permission denied on 'GETDATE' (schema Common)	Authz INSERT a AuthorizationControl / Trace , que tienen DEFAULT Common.GETDATE ()	GRANT EXECUTE ON OBJECT::Common.GETDATE MI del Authz
SELECT permission denied on 'AuthorizationControl' (o dedup falla)	Authz sin SELECT sobre schema Authorization (el writer consulta duplicados)	GRANT SELECT ON SCHEMA::Authorization MI del Authz
Normalizer falla en Upsert Cosmos / Authz siempre "documento Cosmos NO encontrado"	Falta rol data-plane de Cosmos para la MI	Asignar Data Contributor (Normalizer) / Data Reader (Authz) scoped al con (\$4.1)
The INSERT permission was denied on the object 'AdmissionAuthorizerAssignmentLog' (SQL Error 229) — el AuthorizationAdmissionDistributionConsumer reintenta y no completa el mensaje	MI del Authz con SELECT a nivel de schema pero sin INSERT/UPDATE sobre las tablas del agente (el runbook viejo solo daba INSERT/UPDATE a AuthorizationControl / Trace / ProcessedInbox , no a AdmissionAuthorizerAssignmentLog)	GRANTS de \$4.3 (bloqueo del agente). Los mensajes se procesan solos al a no se pierden
The INSERT permission was denied on the object 'AuthorizationControlAlert' (SQL Error 229) al registrar la alerta de estancia no liquidada	MI del Authz sin INSERT/UPDATE sobre Authorization.AuthorizationControlAlert (tabla de la alerta roja del dashboard)	GRANT INSERT, UPDATE [Authorization].[AuthorizationControlAlert] a la MI del Authz (\$4.3)
TenantNotActiveException para un tenant que no reconocés	Evento de OTRO entorno en el bus compartido	No es tuyo; el código completa/ignora. Si es registralo ACTIVE en TenantCatalog
Mensajes acumulados en la suscripción, consumer nunca corre, DeliveryCount=0 sostenido, sin error en logs	ServiceBusTrigger "muerto" (paquete de deployment desincronizado)	Redeploy completo azure functionapp p <app> --dotnet-isol
Admission facts no se publican tras redeploy	Falta rol RBAC Data Sender de la MI del Normalizer sobre clinical-admission-facts	Asignar el rol scoped
Órdenes que "a veces llegan y a veces no" a integraciones	Consumidores en competencia sobre la misma suscripción (más de una instancia leyendo la misma suscripción: cada mensaje se lo lleva UNO solo)	Que solo la Function A desplegada del ambiente consuma cada suscripción producción
Dashboard: columna Cantidad vacía en Atención Inicial de Urgencias	RequestedQuantity quedaba NULL en el flujo AIU	RequestedQuantity= defecto + backfill al re
Unidad Funcional de la orden no corresponde (sale la de Urgencias)	Se tomaba ADINGRESO.UFUCODIGO (del ingreso) en vez de HCHISPACA.UFUCODIGO (del folio)	Normalizer propaga functionalUnitCode Authz lo prefiere sobre ingreso

7. Referencias

- Runbook SQL de onboarding: [VieCloud_DB_Colombia_Schema/Vie_Erp/Authorization/Scripts/ERP84_OnboardingRunbook.sql](#)
- Grants del agente: [Deploy_BD636_DistribucionAutorizadores.sql](#) (Partes 1.2 bis / 1.4 bis)
- laC Service Bus: [EHR_AzureFunctions/deploy/bicep/modules/servicebus.bicep](#)

- Deploy / RBAC / app settings: [EHR_AzureFunctions/deploy/README.md](#)